Absolutely — this time use a **Natural Language Processing (NLP)** document. It will give you a different type of RAG test because it contains definitions, algorithms, formulas, comparisons, numerical facts, and multi-hop relationships.

Natural Language Processing Fundamentals

A Structured Technical Document for RAG Evaluation

Chapter 1: Introduction to Natural Language Processing

Natural Language Processing (NLP) is a field of artificial intelligence that focuses on enabling computers to process, understand, analyze, and generate human language.

NLP combines concepts from:

- Artificial intelligence
- Machine learning
- Deep learning
- Linguistics
- Information retrieval

Common NLP applications include:

- Chatbots
- Search engines
- Sentiment analysis
- Machine translation
- Text classification
- Question answering
- Information extraction
- Document summarization
- Named entity recognition

For example, a customer-support chatbot may receive:

> "My order hasn't arrived yet."

The system can identify the user's intent as **order-status inquiry** and extract relevant information such as the order number if it is provided.

Chapter 2: Text Processing Pipeline

A typical NLP pipeline contains several stages:

Raw Text → Cleaning → Tokenization → Normalization → Feature Representation → Model → Output

Different applications may use different subsets of these steps.

For example, a sentiment-analysis system could use:

Review → Tokenization → Embedding → Classification Model → Sentiment

A document-retrieval system could use:

Document → Chunking → Embedding → Vector Database → Retrieval

---

Chapter 3: Tokenization

Tokenization divides text into smaller units called tokens.

Consider the sentence:

> "Machine learning is powerful."

A simple word tokenizer might produce:

```
["Machine", "learning", "is", "powerful"]
```

Depending on the tokenizer, punctuation may be treated as a separate token.

Modern transformer tokenizers may use **subword tokenization**.

For example, an uncommon word can potentially be divided into smaller pieces instead of being treated as one unknown word.

Common subword approaches include:

- Byte Pair Encoding (BPE)
- WordPiece
- SentencePiece

Subword tokenization helps models handle rare and previously unseen words.

Chapter 4: Stop Words

Stop words are frequently occurring words that may contribute relatively little information for some NLP tasks.

Examples can include:

- the
- is
- a
- an
- of
- and

Removing stop words can reduce the size of a text representation.

However, stop-word removal is not universally beneficial.

For sentiment analysis, removing a word such as **"not"** could change the meaning of a sentence.

Consider:

> "The product is not good."

If "not" is removed, the resulting text could incorrectly appear to express a positive sentiment.

Therefore, preprocessing should depend on the task.

Chapter 5: Stemming and Lemmatization

Stemming and lemmatization attempt to reduce words to a simpler base form.

5.1 Stemming

Stemming applies heuristic rules to remove parts of words.

For example:

- playing → play
- played → play
- studies → studi

Stemming is generally faster but may produce linguistically incorrect results.

5.2 Lemmatization

Lemmatization attempts to return a valid dictionary form.

For example:

- running → run
- better → good
- studies → study

Lemmatization usually requires more linguistic information than simple stemming.

Chapter 6: Part-of-Speech Tagging

Part-of-Speech (POS) tagging assigns grammatical categories to words.

Common POS categories include:

- Noun
- Verb
- Adjective
- Adverb
- Pronoun
- Preposition
- Conjunction

Consider:

> "The engineer built a model."

Possible tags include:

- engineer → noun
- built → verb
- model → noun

POS tagging can be useful for information extraction and linguistic analysis.

Chapter 7: Named Entity Recognition

Named Entity Recognition (NER) identifies entities in text and assigns them categories.

Common entity types include:

- PERSON
- ORGANIZATION
- LOCATION
- DATE
- MONEY
- PRODUCT

For example:

"Elon Musk founded SpaceX in 2002."

A simplified NER system might identify:

- Elon Musk → PERSON
- SpaceX → ORGANIZATION
- 2002 → DATE

NER is useful for:

- Information extraction
- Search
- Document analysis
- Question answering
- Knowledge graph construction

Chapter 8: Bag of Words

Bag of Words (BoW) represents text based on word occurrence.

Suppose we have two documents:

Document 1:
"Machine learning is useful."

Document 2:
"Machine learning is powerful."

The vocabulary could be:

```
Machine, learning, is, useful, powerful
```

The documents can then be represented as numerical vectors.

For example:

Document 1:

```
[1, 1, 1, 1, 0]
```

Document 2:

```
[1, 1, 1, 0, 1]
```

BoW is simple but ignores word order and deeper semantic relationships.

For example:

"Dog bites man."

and

"Man bites dog."

could produce similar word-frequency representations despite having different meanings.

Chapter 9: TF-IDF

TF-IDF stands for **Term Frequency-Inverse Document Frequency**.

It measures how important a word is to a document relative to a collection of documents.

TF-IDF contains two main components:

- Term Frequency (TF)
- Inverse Document Frequency (IDF)

A common formulation is:

TF-IDF(t,d) = TF(t,d) × IDF(t)

The IDF component can be expressed as:

IDF(t) = log(N / df(t))

where:

- N = total number of documents
- df(t) = number of documents containing term t

A word appearing frequently in one document but rarely across the entire collection can receive a high TF-IDF value.

Common words appearing in nearly every document tend to have lower IDF values.

Chapter 10: Word Embeddings

Word embeddings represent words as dense numerical vectors.

Instead of representing a word as a sparse one-hot vector, an embedding maps it into a continuous vector space.

For example:

king → [0.21, -0.14, 0.67, ...]

queen → [0.19, -0.12, 0.64, ...]

Words with similar meanings can have similar vector representations.

Embeddings can capture semantic relationships that simple word-count representations cannot.

Chapter 11: Word2Vec

Word2Vec is an embedding technique that learns word representations from surrounding context.

Two common Word2Vec architectures are:

1. Continuous Bag of Words (CBOW)
2. Skip-Gram

CBOW

CBOW predicts a target word using surrounding context words.

For example:

"The cat ___ on the mat."

The model attempts to predict:

sat

Skip-Gram

Skip-Gram performs the opposite task.

Given a target word, it attempts to predict surrounding context words.

For example:

Input:

cat

Potential context:

- the
- sat
- on
- mat

Word2Vec is based on the idea that words appearing in similar contexts tend to have similar meanings.

Chapter 12: Sentence Embeddings

Word embeddings represent individual words, while sentence embeddings represent larger pieces of text.

A sentence embedding converts a sentence or paragraph into a vector.

For example:

"The car is very fast."

and

"The vehicle has high speed."

may receive relatively similar embeddings because they have similar semantic meanings.

Sentence embeddings are particularly useful for:

- Semantic search
- Document retrieval
- Clustering
- Duplicate detection
- Recommendation systems
- RAG systems

Chapter 13: Semantic Similarity

Semantic similarity measures how closely two pieces of text are related in meaning.

A commonly used metric for vector representations is **cosine similarity**.

The formula is:

cos(θ) = (A · B) / (||A|| ||B||)

The value generally ranges from:

-1 to 1

For many embedding applications, a higher cosine similarity indicates greater semantic similarity.

Consider:

Sentence A:

"I enjoy programming."

Sentence B:

"I like writing code."

These sentences may have high semantic similarity despite using different words.

Chapter 14: Sequence Models

Some NLP tasks require understanding word order and sequential relationships.

Traditional sequence models include:

- RNN
- LSTM
- GRU

RNNs process sequences one step at a time and maintain a hidden state.

LSTMs introduce gates to better preserve useful information over longer sequences.

The three major LSTM gates are:

1. Forget gate
2. Input gate
3. Output gate

GRU is another recurrent architecture that uses fewer gates than LSTM.

Chapter 15: Attention Mechanism

Attention allows a model to assign different importance to different parts of an input.

Consider:

> "The student who studied machine learning passed the exam because she practiced regularly."

When processing **"she"**, a model may need to determine that **"student"** is the relevant earlier word.

Attention allows models to dynamically determine which tokens are important for a particular prediction.

The standard scaled dot-product attention formula is:

$$\textbf{Attention}(Q,K,V) = \textbf{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where:

- Q = Query
- K = Key
- V = Value
- d_k = key dimension

Chapter 16: Transformers

Transformers use attention mechanisms instead of relying primarily on recurrent processing.

A simplified Transformer block includes:

Self-Attention → Add & Normalize → Feed-Forward Network → Add & Normalize

Transformers can process relationships between tokens efficiently and have become foundational to modern NLP systems.

Examples of Transformer-based model families include:

- BERT
- GPT
- T5

BERT

BERT is designed primarily for understanding language representations.

It uses bidirectional context and was originally pretrained using tasks including masked language modeling.

GPT

GPT-style models are autoregressive language models.

They generate text by predicting the next token based on previous tokens.

T5

T5 treats many NLP tasks as text-to-text problems.

For example:

Input: Translate English to French: "Hello."

Output: "Bonjour."

Chapter 17: Text Classification

Text classification assigns one or more categories to a text.

Examples include:

- Spam detection
- Sentiment classification
- Topic classification
- Intent detection
- Toxicity detection

A classification pipeline might be:

Text → Tokenization → Representation → Classifier → Class

For example:

> "I cannot access my account."

could be classified as:

Account Access Problem

Chapter 18: Sentiment Analysis

Sentiment analysis identifies the emotional or opinion-based polarity of text.

Common categories are:

- Positive
- Negative
- Neutral

Example:

> "The camera quality is excellent."

Sentiment:

Positive

Another example:

> "The battery stopped working after two days."

Sentiment:

Negative

Sentiment analysis can be applied to:

- Product reviews
- Social media
- Customer feedback
- Surveys
- Support tickets

Chapter 19: Question Answering

Question Answering (QA) systems generate or retrieve answers to user questions.

A QA system may operate in two major ways.

Extractive QA

The answer is extracted directly from a provided document.

Example:

Document:

> "The company was founded in 2015."

Question:

> "When was the company founded?"

Answer:

2015

Generative QA

A language model generates an answer based on learned information or retrieved context.

Generative QA can produce natural-language responses but requires grounding mechanisms when factual accuracy is important.

Chapter 20: Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) combines information retrieval with language generation.

A basic RAG pipeline is:

Documents → Chunking → Embeddings → Vector Database → Retrieval → Context → LLM → Answer

20.1 Document Ingestion

Documents are first processed and divided into smaller chunks.

Each chunk receives an embedding.

The embeddings are stored in a vector database.

20.2 Query Processing

When a user asks a question, the query is converted into an embedding.

The system searches for chunks that are semantically similar to the query.

20.3 Context Construction

The most relevant retrieved chunks are passed to the language model.

The LLM generates an answer using the retrieved information.

RAG can reduce the need for a model to rely entirely on information stored in its parameters.

Chapter 21: Practical RAG Case Study

A university creates a RAG system for answering questions about its academic regulations.

The system contains **2,400 PDF documents**.

After processing, the documents produce:

18,600 text chunks

Each chunk is converted into a **768-dimensional embedding**.

The embeddings are stored in a vector database.

For each query, the system initially retrieves the **top 10 chunks**.

A reranker then selects the **top 4 chunks**.

These four chunks are passed to the language model as context.

21.1 Example Query

User:

> "How many credit hours are required for graduation?"

The retrieval system returns four relevant chunks.

The language model generates an answer using these chunks.

21.2 Evaluation

The team evaluates 100 test questions.

Results:

Metric	Score
Retrieval Recall@10	92%
Retrieval Precision@4	84%
Answer Accuracy	88%
Citation Accuracy	91%

The system performs well overall, but the lower Precision@4 indicates that some retrieved chunks are irrelevant.

Chapter 22: NLP Model Comparison

Technique	Representation	Main Use	Main Limitation
Bag of Words	Sparse	Basic text classification	Ignores word order
TF-IDF	Sparse weighted	Search/classification	Limited semantic understanding
Word2Vec	Dense word vectors	Word similarity	Context is limited
Sentence Embeddings	Dense sentence vectors	Semantic search	Quality depends on embedding model
RNN	Sequential hidden states	Sequence processing	Sequential computation
LSTM	Gated hidden states	Long dependencies	More complex
Transformer	Attention-based	Modern NLP	Computationally expensive

---

Chapter 23: Important Formulas

TF-IDF

$$\text{TF-IDF}(t,d) = \text{TF}(t,d) \times \text{IDF}(t)$$

IDF

$$\text{IDF}(t) = \log\left(\frac{N}{df(t)}\right)$$

Cosine Similarity

$$\cos(\theta) = \frac{A \cdot B}{||A||\,||B||}$$

Attention

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Accuracy

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}}$$

Precision

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall

$$\text{Recall} = \frac{TP}{TP + FN}$$

Chapter 24: Glossary

Embedding: A numerical vector representation of information.

Entity: A person, organization, location, product, date, or other identifiable object.

IDF: A measure indicating how rare a term is across a document collection.

NER: Named Entity Recognition.

NLP: Natural Language Processing.

RAG: Retrieval-Augmented Generation.

Semantic Search: Search based on meaning rather than only exact keyword matching.

Stop Word: A frequently occurring word that may be removed during some preprocessing tasks.

Token: A unit of text processed by an NLP model.

Tokenizer: A system that converts text into tokens.

Transformer: A neural-network architecture based heavily on attention.

Vector Database: A database optimized for storing and searching vector representations.
